SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 0614 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning ***(BL 5)***

Assessment Tool Weightage: 10%

QUESTIONS: 5 **Total Marks:50**

Annexure A

Comparative Analysis

Code of Conduct:

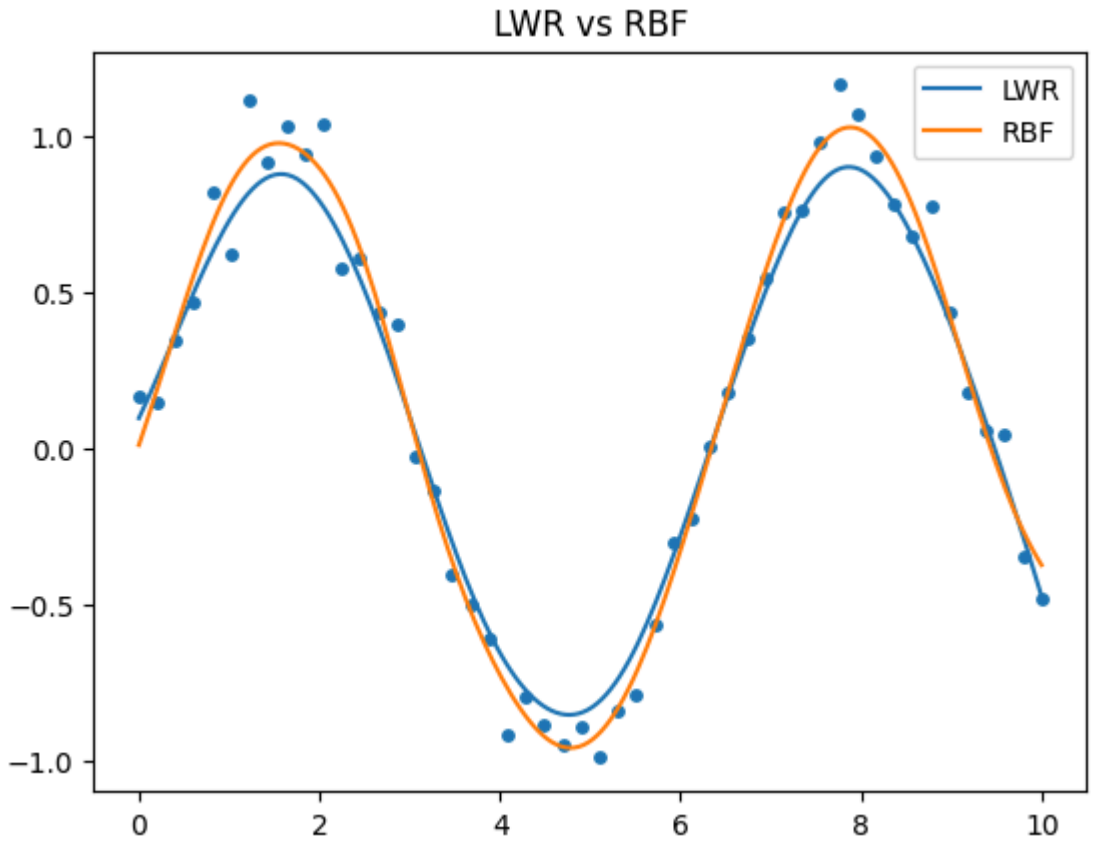
I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

2.
```
import numpy as np, matplotlib.pyplot as plt
from sklearn.cluster import KMeans

np.random.seed(1)
X=np.linspace(0,10,50); y=np.sin(X)+np.random.randn(50)*.1
Xt=np.linspace(0,10,200)

def lwr(x):
    w=np.exp(-(X-x)**2/(2*.5**2))
    A=np.c_[np.ones(len(X)),X]
                                                                return
np.array([1,x]@np.linalg.pinv(A.T@np.diag(w)@A)@A.T@np.diag(w)@y)

YL=np.array([lwr(x) for x in Xt])

C=KMeans(10,n_init=10,random_state=1).fit(X[:,None]).cluster_centers_.r
avel()
R=lambda z:np.exp(-(z[:,None]-C)**2/(2*.7**2))
W=np.linalg.pinv(np.c_[np.ones(50),R(X)])@y
YR=np.c_[np.ones(200),R(Xt)]@W

plt.scatter(X,y,s=15); plt.plot(Xt,YL,label="LWR")
plt.plot(Xt,YR,label="RBF"); plt.legend(); plt.title("LWR vs RBF")
plt.show()
```



3.import numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score

X,y=make_moons(300,noise=.2,random_state=1)
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.3,random_state=1)

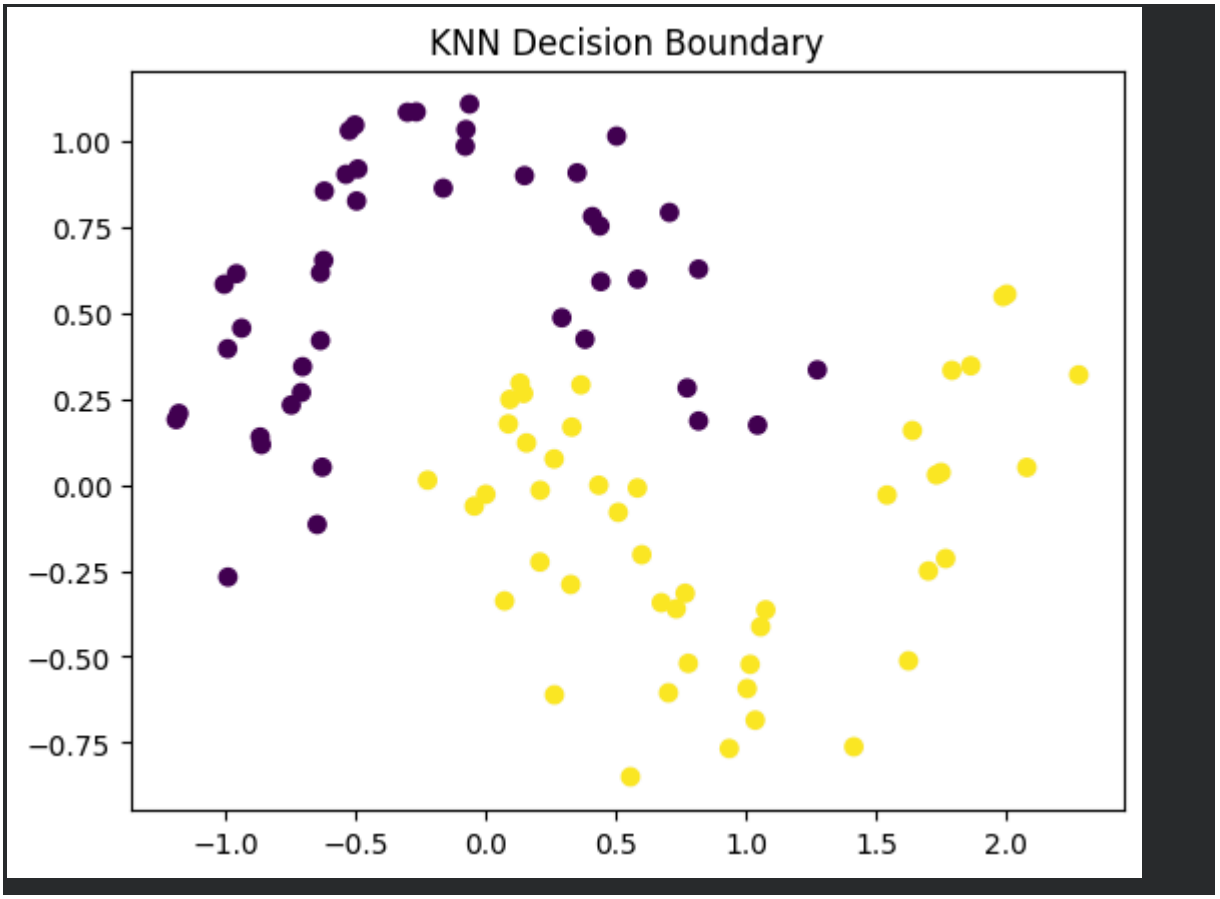
knn=KNeighborsClassifier(5).fit(Xtr,ytr)
p1=knn.predict(Xte)

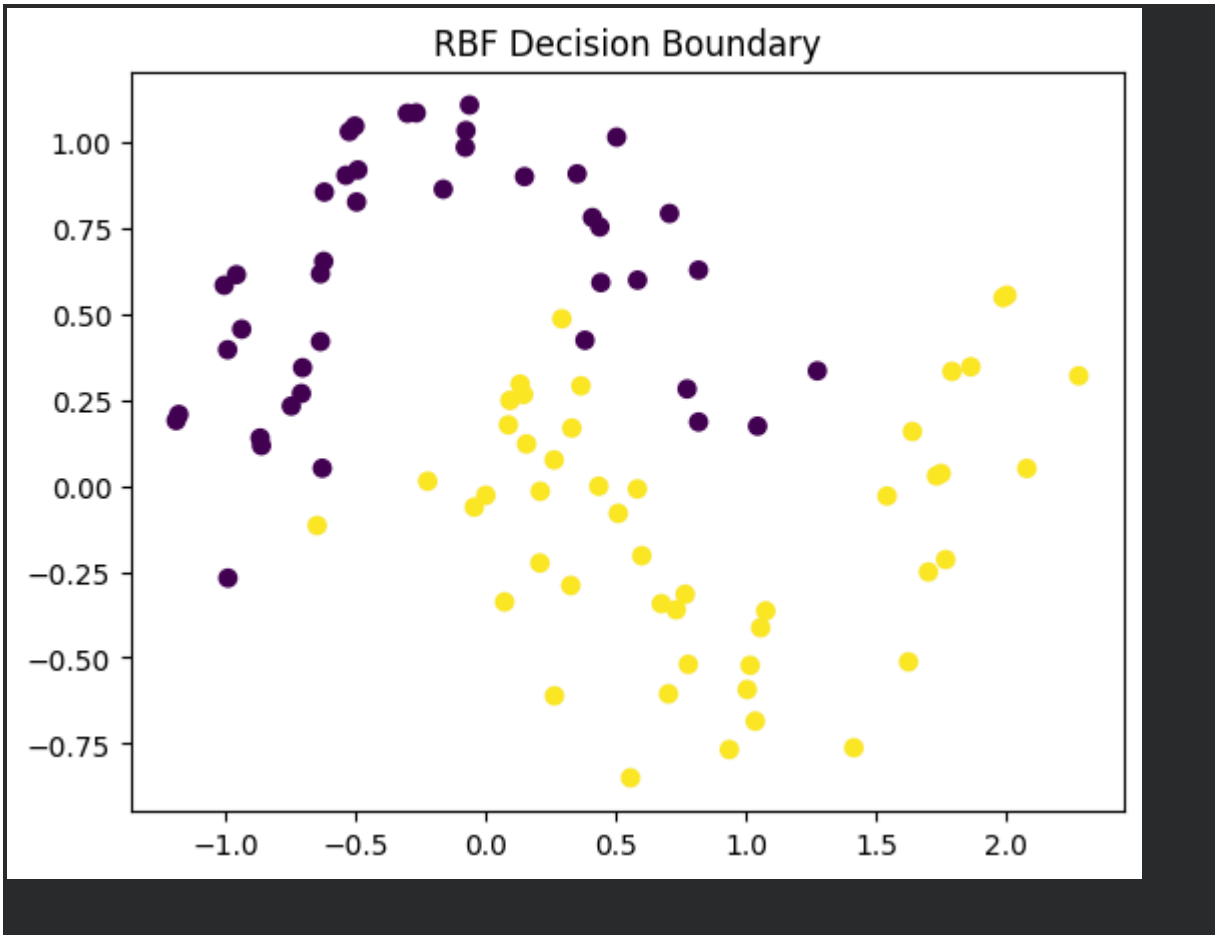
C=KMeans(15,n_init=10,random_state=1).fit(Xtr).cluster_centers_
R=lambda z:np.exp(-((z[:,None]-C)**2).sum(2)/(2*.8**2))
W=np.linalg.pinv(np.c_[np.ones(len(Xtr)),R(Xtr)])@np.where(ytr==0,-1,1)
p2=(np.c_[np.ones(len(Xte)),R(Xte)]@W>=0).astype(int)

print("KNN Accuracy:",accuracy_score(yte,p1))
print("RBF Accuracy:",accuracy_score(yte,p2))

plt.scatter(Xte[:,0],Xte[:,1],c=p1); plt.title("KNN Decision Boundary"); plt.show()
plt.scatter(Xte[:,0],Xte[:,1],c=p2); plt.title("RBF Decision Boundary"); plt.show()

KNN Accuracy: 0.9777777777777777
RBF Accuracy: 0.9777777777777777

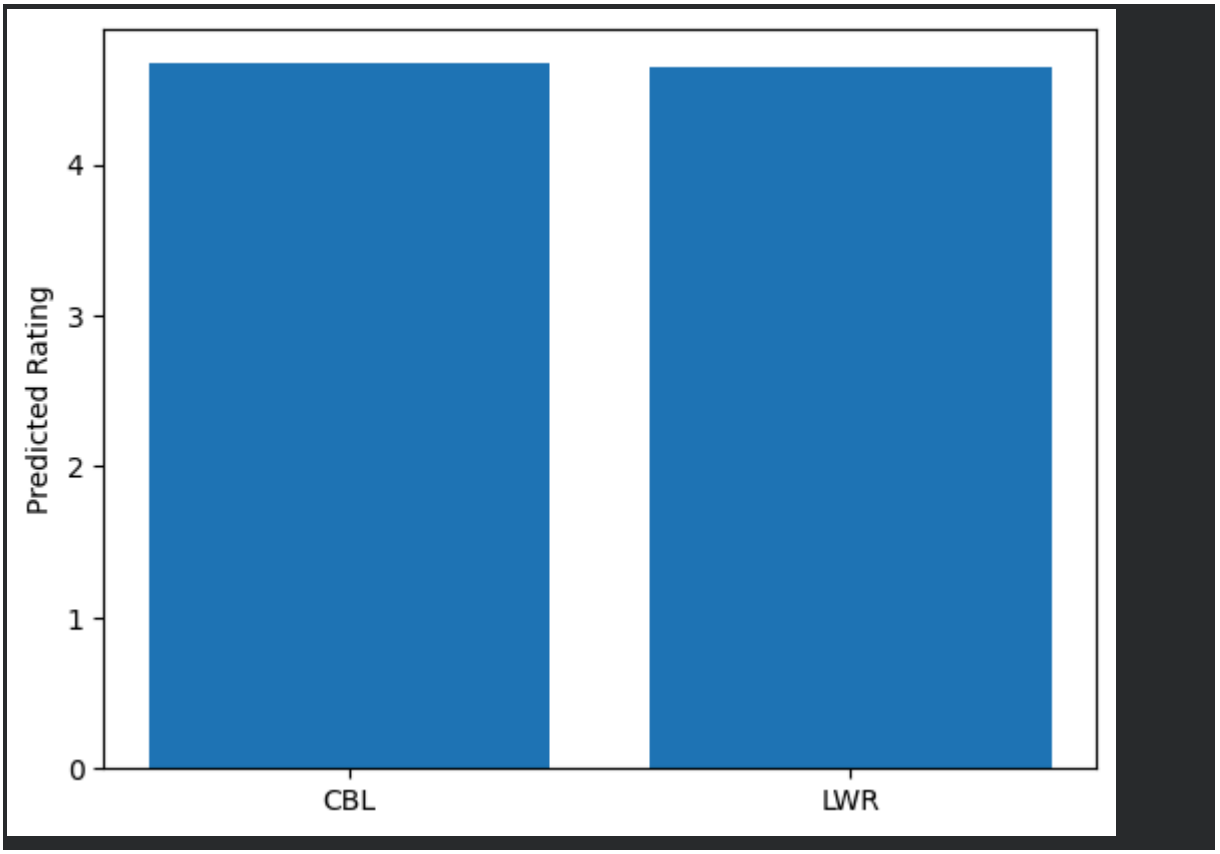




4.import numpy as np, matplotlib.pyplot as plt

```
# User ratings: Inception, Interstellar, Avatar, Titanic, Target
D=np.array([
[5,5,4,2,5],[5,4,5,2,5],[4,5,4,3,4],
[2,2,3,5,2],[5,5,4,2,5],[4,4,5,3,4]
])

new=np.array([5,5,4,2])
d=np.linalg.norm(D[:,:4]-new,axis=1)

# Case-Based Learning
p_cbl=D[np.argsort(d)[:3],4].mean()

# Locally Weighted Regression
w=np.exp(-d**2/(2*2**2))
p_lwr=(w*D[:,4]).sum()/w.sum()

print("CBL Prediction:",round(p_cbl,2))
print("LWR Prediction:",round(p_lwr,2))

plt.bar(["CBL","LWR"],[p_cbl,p_lwr])
```

plt.ylabel("Predicted Rating")
plt.show()
CBL Prediction: 4.67
LWR Prediction: 4.65



6.import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

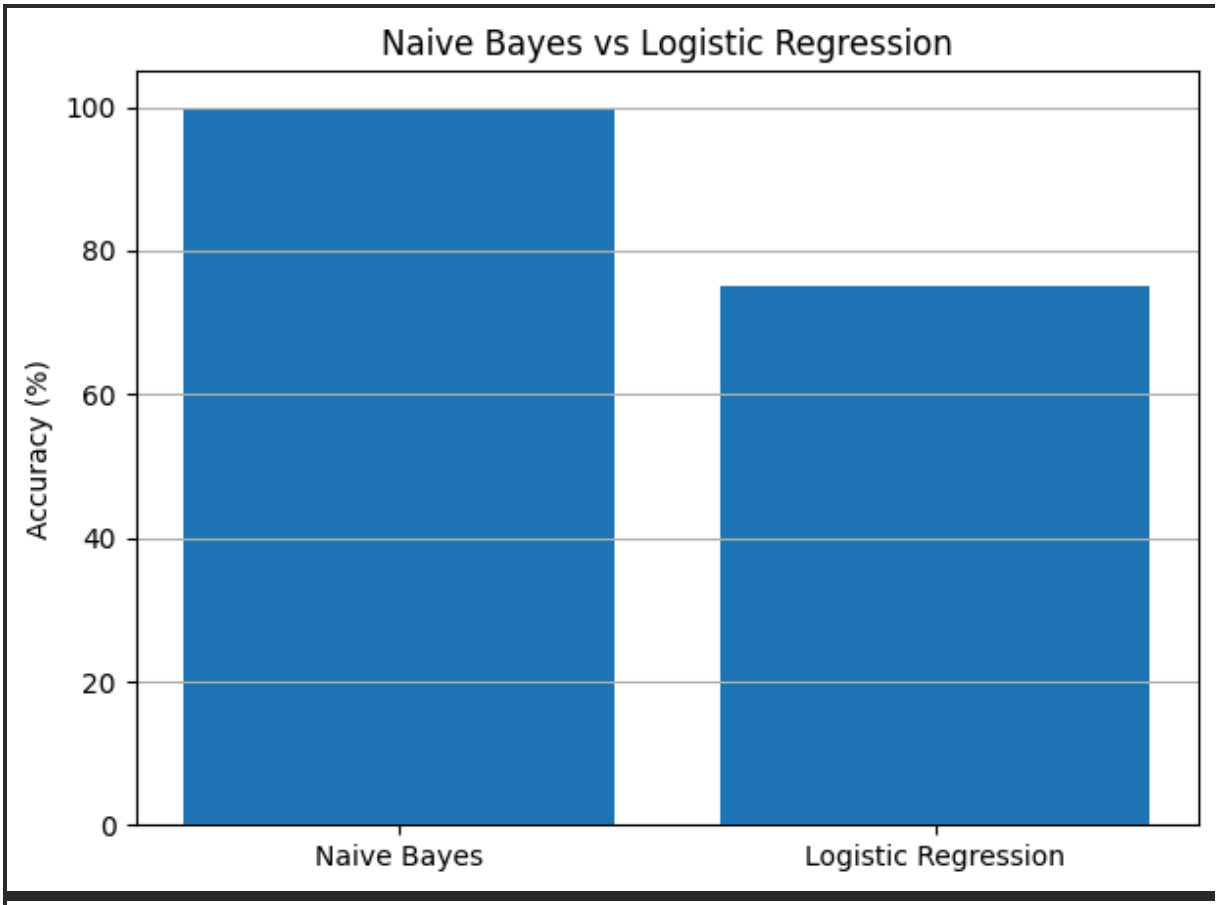
text=[
"win free money","free lottery prize","claim free gift","win cash now",
"free bonus offer","click to win","meeting tomorrow","send project report",
"class starts tomorrow","submit assignment","project meeting","send notes"
]
y=["spam"]*6+["ham"]*6

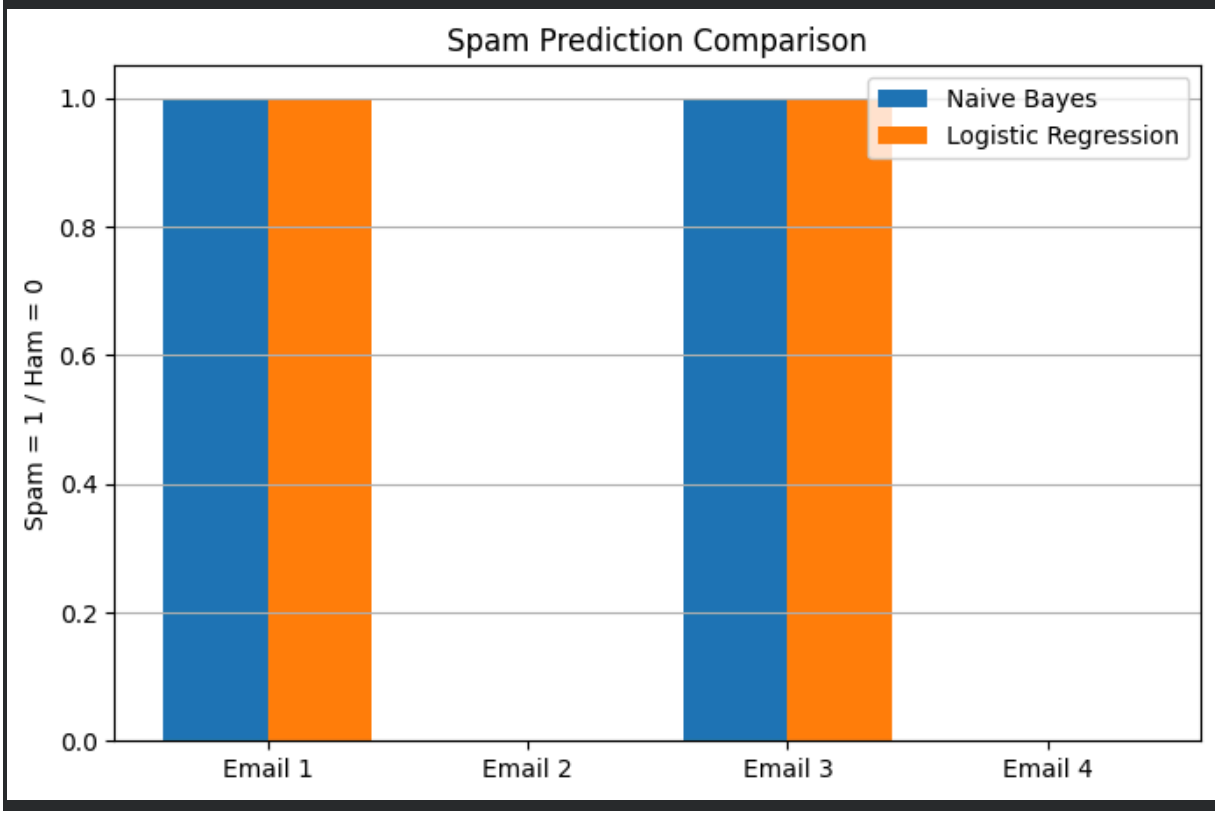
v=TfidfVectorizer(ngram_range=(1,2))
X=v.fit_transform(text)

a,b,c,d=train_test_split(
X,y,test_size=.3,random_state=1,stratify=y
)

```
nb=MultinomialNB().fit(a,c)
lr=LogisticRegression().fit(a,c)

p1=nb.predict(b)
p2=lr.predict(b)

nb_acc=accuracy_score(d,p1)*100
lr_acc=accuracy_score(d,p2)*100

print("Naive Bayes Accuracy:",round(nb_acc,2),"%")
print("Logistic Regression Accuracy:",round(lr_acc,2),"%")

# Accuracy Graph
plt.figure(figsize=(7,5))
plt.bar(["Naive Bayes","Logistic Regression"],[nb_acc,lr_acc])
plt.ylabel("Accuracy (%)")
plt.title("Naive Bayes vs Logistic Regression")
plt.ylim(0,105)
plt.grid(axis="y")
plt.show()

# Prediction Graph
new=v.transform([
   "Congratulations you won free money",
   "Please send the project report",
   "Claim your free gift now",
   "Meeting is scheduled tomorrow"
])

nb_pred=nb.predict(new)
lr_pred=lr.predict(new)

plt.figure(figsize=(8,5))
x=range(4)
plt.bar([i-.2 for i in x],
       [1 if p=="spam" else 0 for p in nb_pred],
       width=.4,label="Naive Bayes")
plt.bar([i+.2 for i in x],
       [1 if p=="spam" else 0 for p in lr_pred],
       width=.4,label="Logistic Regression")

plt.xticks(x,["Email 1","Email 2","Email 3","Email 4"])
plt.ylabel("Spam = 1 / Ham = 0")
plt.title("Spam Prediction Comparison")
plt.legend()
plt.grid(axis="y")
plt.show()
```

```
Naive Bayes Accuracy: 100.0 %
Logistic Regression Accuracy: 75.0 %
```



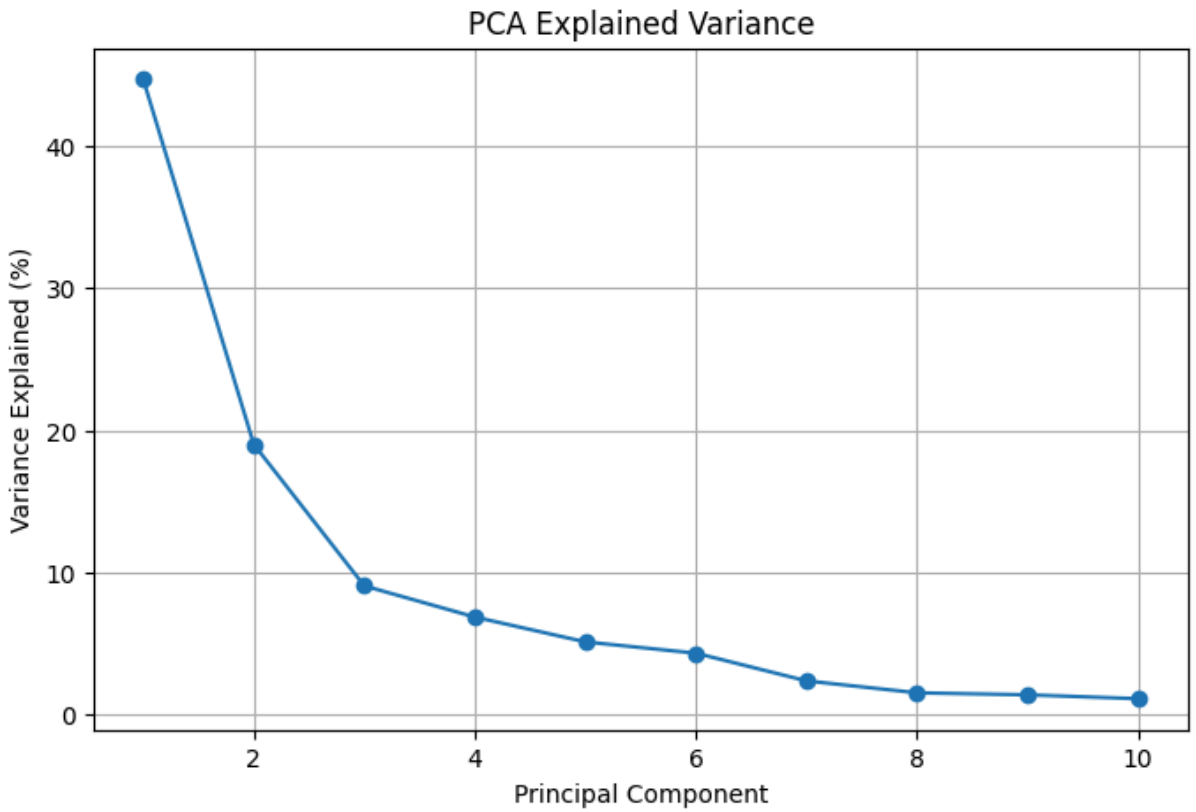


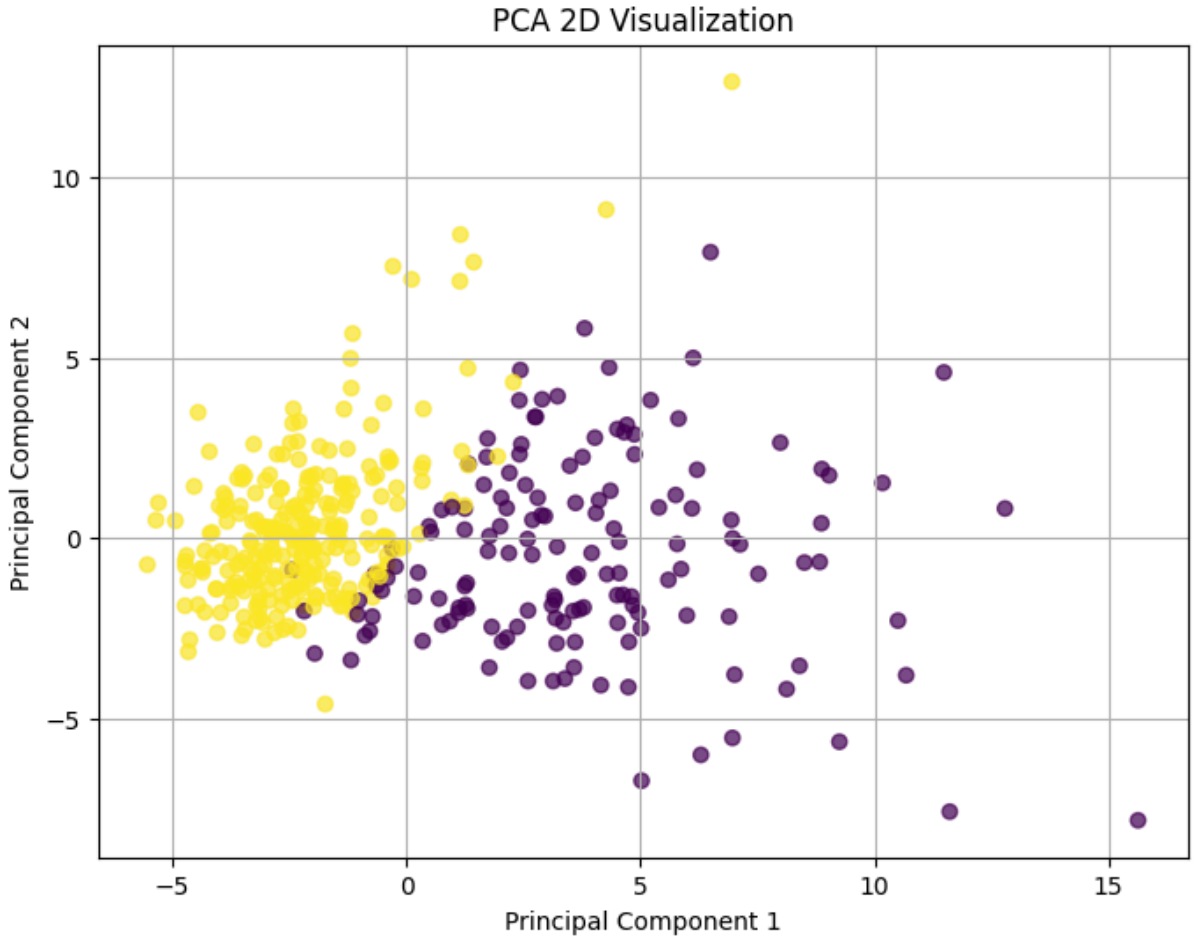
10.import matplotlib.pyplot as plt

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest,f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

D=load_breast_cancer()
X,y=D.data,D.target

a,b,c,d=train_test_split(
   X,y,test_size=.3,random_state=1,stratify=y
)

s=StandardScaler()
a=s.fit_transform(a)
b=s.transform(b)

# PCA
p=PCA(n_components=10)
ap=p.fit_transform(a)
bp=p.transform(b)

# Feature Selection
f=SelectKBest(f_classif,k=10)
af=f.fit_transform(a,c)
bf=f.transform(b)

m1=LogisticRegression(max_iter=3000).fit(ap,c)
m2=LogisticRegression(max_iter=3000).fit(af,c)

pca_acc=accuracy_score(d,m1.predict(bp))*100
fs_acc=accuracy_score(d,m2.predict(bf))*100

print("PCA Accuracy:",round(pca_acc,2),"%")
print("Feature Selection Accuracy:",round(fs_acc,2),"%")
print("PCA Variance:",round(p.explained_variance_ratio_.sum()*100,2),"%")

print("\nSelected Features:")
print(D.feature_names[f.get_support()])

# Accuracy Graph
plt.figure(figsize=(7,5))
plt.bar(["PCA","Feature Selection"],[pca_acc,fs_acc])
plt.ylabel("Accuracy (%)")
plt.title("PCA vs Feature Selection")
plt.ylim(0,105)
plt.grid(axis="y")
plt.show()
```

```
# Number of Features Graph
plt.figure(figsize=(7,5))
plt.bar(
    ["Original","PCA","Feature Selection"],
    [X.shape[1],ap.shape[1],af.shape[1]]
)
plt.ylabel("Number of Features")
plt.title("Dimensionality Reduction Comparison")
plt.grid(axis="y")
plt.show()

# PCA Explained Variance Graph
plt.figure(figsize=(8,5))
plt.plot(
    range(1,11),
    p.explained_variance_ratio_*100,
    marker="o"
)
plt.xlabel("Principal Component")
plt.ylabel("Variance Explained (%)")
plt.title("PCA Explained Variance")
plt.grid(True)
plt.show()

# PCA 2D Visualization
p2=PCA(n_components=2)
z=p2.fit_transform(a)

plt.figure(figsize=(8,6))
plt.scatter(z[:,0],z[:,1],c=c,alpha=.7)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA 2D Visualization")
plt.grid(True)
```

plt.show()


PCA Explained Variance
Variance Explained (%)
Principal Component
0
10
20
30
40
2
4
6
8
10


PCA 2D Visualization
Principal Component 2
Principal Component 1
10
5
0
−5
−5
0
5
10
15

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**15**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**20**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand